Design and Operation of permbox-fuse3

A persistent policy trie and an embeddable FUSE 3 filesystem

permbox-fuse3 project

26 July 2026

Abstract

permbox-fuse3 is a Linux library that exposes a backing directory through a low-level FUSE 3 mount and applies path-based access rules. The project exports two Zig modules. `permtrie` stores rules in a memory-mapped radix trie. `permfuse` connects that trie to FUSE lookup, open, read, write, and directory operations. Fully allowed files can use kernel FUSE passthrough. Restricted writes can be stored in a sparse overlay session and applied to the backing filesystem later. This paper describes the data layout, lookup rules, locking, file handle state, overlay journal, recovery procedure, text format, and public integration points.

Contents

1 Scope and goals

The library is the filesystem component used by permbox. An installed interactive command also provides direct mounts and live policy updates. An embedding caller can create a driver, load or update policy, mount the filesystem, handle permission questions, and apply overlay data when the mount has stopped.

The design has five main goals:

1. Keep common read and write paths short.
2. Store policy in a persistent binary form that is cheap to query.
3. allow policy to become less restrictive while files are open.
4. use kernel passthrough when a handle needs no further policy checks.
5. keep overlay writes recoverable across process exits and partial apply operations.

The implementation targets Linux, glibc, libfuse 3.18, and Zig's `std.Io` synchronization primitives. FUSE over `io_uring` is requested by default. The fallback path remains a normal multithreaded low-level FUSE loop.

2 Module and process structure

Figure 1 shows the main components. The embedding application owns the process and calls the public API. The kernel sends filesystem requests through libfuse. Policy and overlay state remain separate from the backing directory.

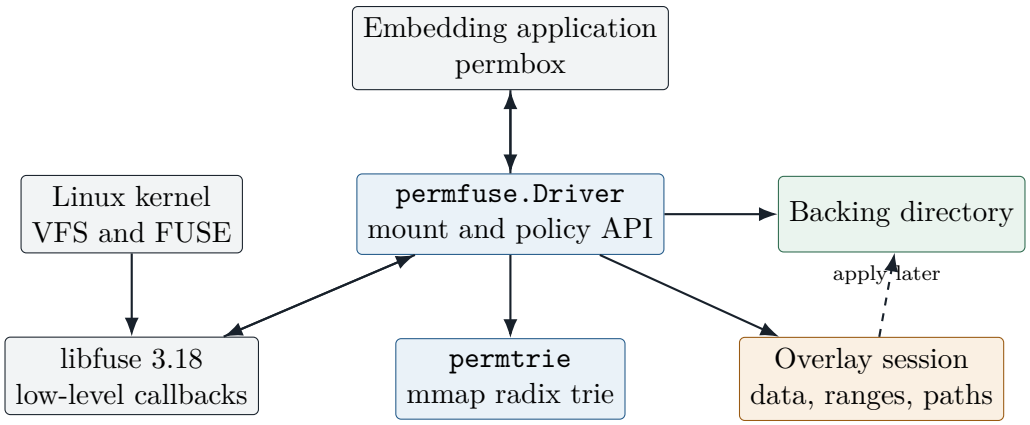

Figure 1: Main components and data paths.

2.1 `permtrie`

`permtrie` is the public, locked wrapper around the raw trie. It owns a file descriptor, a shared memory mapping, and an `std.Io.RwLock`. Its public operations provide exact lookup, inherited lookup, mutation, grouped mutation, reset, and text conversion.

2.2 `permfuse`

`permfuse` owns the active policy trie, the backing directory handle, optional overlay session state, and the FUSE callback state. Only one driver is active in a process at a time. The driver is initialized in its final storage location. It must not be moved while mounted because libfuse stores a pointer to its callback state.

3 Policy representation

3.1 One-byte modes

Each explicit rule stores a packed one-byte mode. The byte contains four two-bit fields:

Field	Type	Values	Meaning
`k`	`Mode.K`	`midway`, `visible_raw`, `visible_virtual`, `invisible`	Namespace kind and source of file contents.
`r`	`Mode.A`	`deny`, `ask`, `allow`, reserved	Read and open access.
`w`	`Mode.W`	`deny`, `ask`, `allow`, `overlay`	Write behavior and destination.
`x`	`Mode.A`	`deny`, `ask`, `allow`, reserved	Execute access.

Table 1: Fields in a trie rule.

`midway` is used by internal radix nodes that have no explicit rule. It is not a valid serialized policy entry. `visible_raw` refers to the backing filesystem. `visible_virtual` describes a path whose content must not come from the backing filesystem. `invisible` hides a path.

3.2 Exact and inherited lookup

An exact lookup returns only a rule stored at the requested path. An effective lookup first checks that exact path, then removes one slash-delimited component at a time until it reaches `/`. This makes inheritance follow filesystem components rather than arbitrary byte prefixes.

For example, if `/home` has a rule and `/home/user/file` does not, the effective lookup returns the `/home` rule. A rule for `/hom` does not apply to `/home`.

4 Binary trie layout

4.1 Memory-mapped pool

The binary file is mapped with `MAP_SHARED`. Its allocation unit is 1 KiB. Index zero contains the pool header. Live nodes start at index one. The header records:

- a 14-byte magic value,
- file endianness,
- format version,
- the next sequential free index,
- the head of the free list,
- the root node index.

The initial mapping is 64 KiB. When the sequential frontier reaches the end, the backing file grows and `mremap` may move the mapping. Code that allocates a node therefore stores node indices, not stable pointers. Every pointer that may cross a resize point is fetched again from its index.

The header records file endianness. If a valid trie uses the opposite byte order, initialization converts it through a temporary memory file, copies the native-endian result back to the original mapping, and syncs it before normal validation continues.

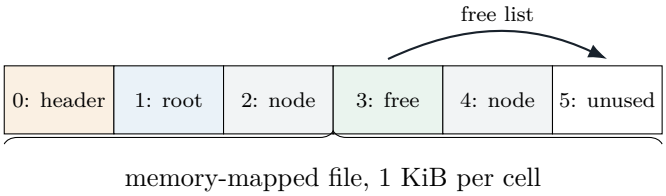

Figure 2: Pool organization. Index zero is never a live trie node.

4.2 Node structure

Each 1 KiB node contains:

- a radix length and radix byte string,
- one mode byte for the path represented by the node,
- a 256-bit child mask,
- 256 packed 24-bit slots.

The child mask changes the meaning of each 24-bit slot. If bit b is set, slot b is a node index. If bit b is clear, slot b is either zero or an inline mode value. Short leaves can therefore store a full rule without a second 1 KiB node.

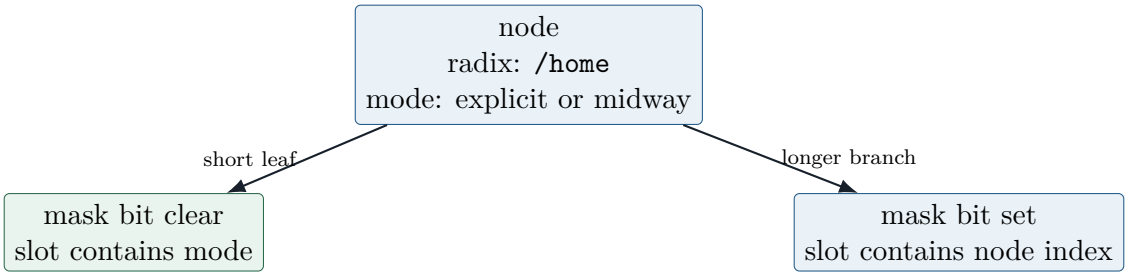

Figure 3: Dual use of a node's 24-bit slots.

4.3 Insertion

Insertion walks the radix strings until one of four cases occurs:

1. Exact match: replace the node mode.
2. Path ends at an inline slot: replace the inline mode.
3. Missing branch: create a compressed chain for the remaining bytes.
4. Radix difference: split the current node into a new parent and two branches.

New nodes are unpublished until every required field is ready. Error cleanup may release only these unpublished nodes. Once a node index is stored in the root or in a published parent, later cleanup must not free it. A failed non-critical cleanup is logged.

4.4 Deletion

Deletion clears an exact mode or inline value, then examines the path back toward the root. Empty nodes can be released. A node can be merged only when the merge preserves lookup behavior. In particular, a zero-length radix node is not merged, and a remaining child with its own children or inline entries is not folded into its parent.

The free list uses two packed slots in a released node. Releasing the last sequential node moves the frontier backward. Releasing any other node adds it to the linked free list.

5 Trie concurrency and persistence

The public trie wrapper uses one read-write lock:

- `get`, `getExact`, and `toText` take a shared lock.
- `add`, `del`, `reset`, and text replacement take an exclusive lock.
- grouped updates use `lockWrite` and the `Locked` operations.

The raw trie is not concurrency safe. The locked wrapper is the supported public interface. A group lock blocks all readers and writers, so another thread cannot observe half of a policy update.

Mutations request an `msync` after committing. A sync failure is logged as a persistence error. It is not returned after the mapped data has changed. An error in a grouped update does not roll back earlier changes.

6 Text and binary conversion

The binary trie can be converted to and from a permbox `fs` block. Import accepts nested entries, flat entries, comments, semicolons, quoted path escapes, and combined permission forms such as `allow-rwx`. Export walks explicit trie entries in byte order and writes a canonical, flat `fs` block.

```
fs {
  "/": access,overlay-w,ALWAYS-allow-rx {
    "home/user": empty,allow-rw,deny-x
  }
}
```

The parser builds a complete temporary rule list before taking the write lock. Syntax errors therefore leave the existing trie unchanged. After parsing, the writer lock is held while the old trie is reset and every new rule is inserted.

Text flag	Stored value
`access`	`k = visible_raw`
`empty`	`k = visible_virtual`
`no-access`	`k = invisible`
`deny-r`, `allow-r`, `ALWAYS-allow-r`	read deny, ask, allow
`deny-w`, `allow-w`, `ALWAYS-allow-w`	write deny, ask, allow
`overlay-w`	write overlay
`deny-x`, `allow-x`, `ALWAYS-allow-x`	execute deny, ask, allow

Table 2: Text flag mapping.

7 FUSE namespace and inode model

The driver uses the low-level FUSE API. Kernel inode numbers other than the root are pointer values for allocated `Inode` objects. This removes a global map lookup from callbacks that already have an inode number.

An inode stores:

- an `O_PATH` descriptor for the backing object,
- its normalized absolute policy path,

- kernel lookup references,
- open handle references,
- an optional shared passthrough backing identifier,
- a short per-inode mutex,
- a separate overlay append mutex.

The global string map is used only to publish and reuse inodes by path. Filesystem I/O is never performed while its mutex is held. A forgotten inode is detached only after both its lookup count and open count reach zero. Directory handles increment the same open count as file handles.

7.1 Lookup

Lookup performs these steps:

1. Join the parent path and child name in a fixed 4096-byte buffer.
2. Evaluate the effective policy.
3. Reject hidden or non-raw entries.
4. Open the child with `O_PATH`, `O_NOFOLLOW`, and `O_CLOEXEC`.
5. Read attributes.
6. Publish or reuse the inode under the short global mutex.
7. Reply to the kernel.

The backing open and attribute read happen outside the inode-map mutex.

8 Open handles and policy snapshots

Opening a file evaluates policy once and stores the full mode byte in the handle. Ordinary read, write, and execute checks load this byte atomically. They do not take the trie lock.

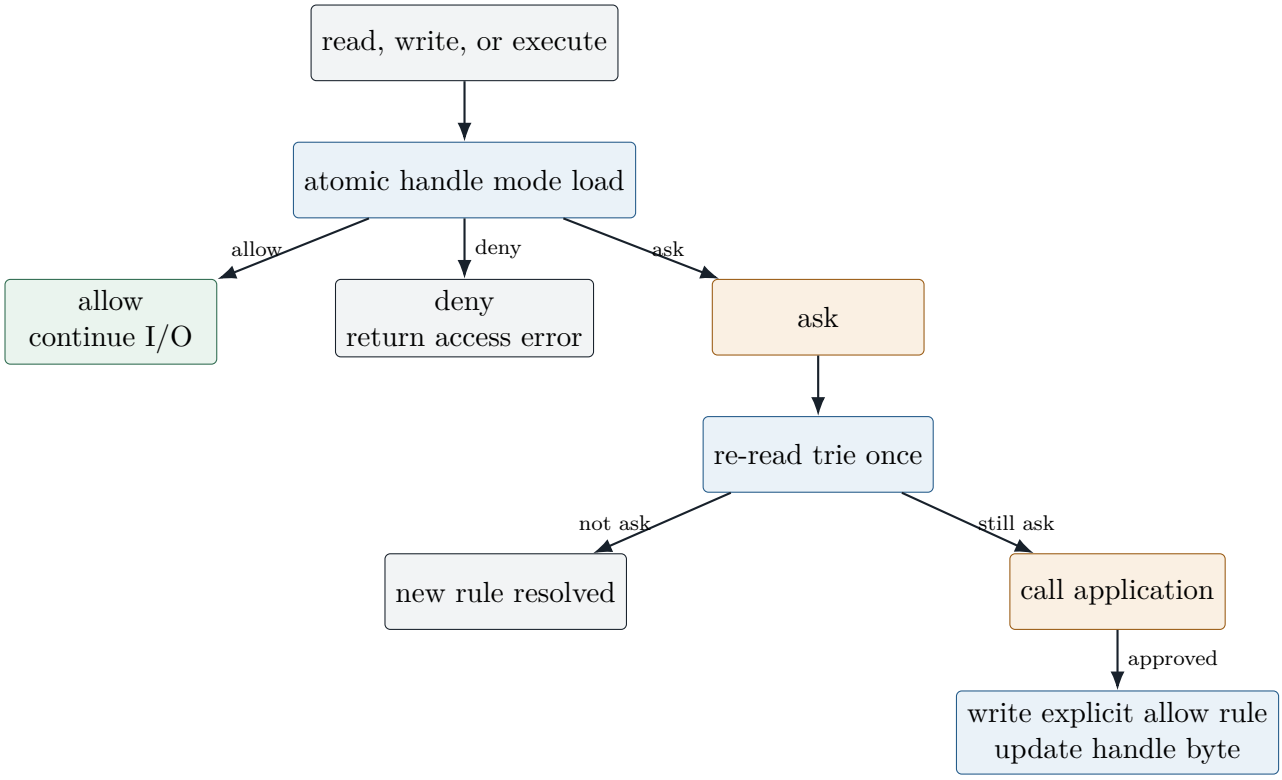

Figure 4: Permission decision for an existing handle.

If the snapshot says `ask`, the handle mutex serializes questions for that open. The trie is read once before the callback. This catches a rule change made by another handle. If the rule still asks, the application callback receives the path, operation, and current mode.

After approval, the code takes the trie writer lock and checks again. If the rule still asks, it changes that operation to allow and writes an explicit rule. It then updates the handle mode. Existing fully allowed handles do not need this path. Existing denied handles remain denied until reopened unless their stored state is `ask`, as defined by the current API.

9 Kernel passthrough

During FUSE initialization, the driver requests `FUSE_CAP_PASSTHROUGH`. If the kernel accepts it, a fully allowed raw inode can register a backing descriptor with libfuse. The resulting backing identifier is placed in `fuse_file_info`.

Passthrough is used only when:

- the path kind is `visible_raw`,
- read is `allow`,
- write is `allow`,
- execute is `allow`,
- the kernel negotiated the capability.

The kernel can then service read, write, splice, and mmap against the backing file without returning to FUSE for each operation. Policy updates do not retroactively change an already opened passthrough descriptor. This matches normal file descriptor behavior.

If passthrough is unavailable, the same handle uses userspace I/O. Policy meaning does not change.

10 Userspace read and write paths

10.1 Raw reads and writes

Raw reads can use `fuse_reply_data` with a file descriptor buffer. Raw writes use positional writes against the open backing descriptor. These operations do not hold the global inode-map mutex.

10.2 Overlay writes

An overlay handle has a sparse data descriptor, a range-journal descriptor, and an in-memory list of range records. A successful positional write to the sparse data file is followed by one fixed-size range record:

$$R = (\mathit{offset} : u64,\ \mathit{length} : u64)$$

The range file is append-only during normal operation. The per-inode overlay mutex prevents records from separate opens of the same inode from interleaving. A torn final record is truncated before further writable use.

Overlay reads construct the logical file as follows:

1. Determine the backing file size.
2. Extend the logical size to the maximum end of any overlay range.
3. Read backing bytes into the reply buffer.

4. Replace intersecting regions with bytes from the sparse overlay file.

Later range records take effect after earlier records because they are applied in journal order.

11 Durable overlay session

A session directory uses this layout:

```
session/
  format
  data/<id>
  ranges/<id>
  paths/<id>
  apply.log
```

The identifier is 32 hexadecimal characters made from two seeded Wyhash values of the normalized absolute path. The hash is only a filename. The `paths` record is the authoritative reverse mapping. Registration checks an existing record byte for byte, so a hash collision is reported instead of writing to the wrong path.

11.1 Publishing a path record

A path record is written to a unique pending file, synced, and linked to its final identifier. The directory is then synced. A process that loses the link race verifies that the winner stored the same path. The final name therefore never points to a partially written path.

11.2 Safe target resolution

Apply opens every intermediate backing directory with `O_NOFOLLOW` and opens the final target with `O_NOFOLLOW`. A stored session path cannot escape the configured backing root through a symlink. Only regular files are accepted as apply targets.

12 Apply protocol and recovery

The apply log stores one line per durable checkpoint:

```
<32-byte-id> <range-journal-offset>
```

For each file, apply reads complete range records starting at the last checkpoint. It copies the described bytes from the sparse data file to the backing target. The order of durability operations is:

1. write all pending ranges to the backing target,
2. `fsync` the target,
3. append the new range offset to `apply.log`,
4. `fdatasync` the apply log.

A crash before the checkpoint can cause the same positional writes to run again. It cannot cause the log to claim that unsynced target data is complete. This makes retry idempotent.

A torn final line in `apply.log` is removed at startup. A partial range record is ignored or truncated to the last complete 16-byte boundary. Missing data, range, or path files are counted as incomplete entries rather than silently accepted.

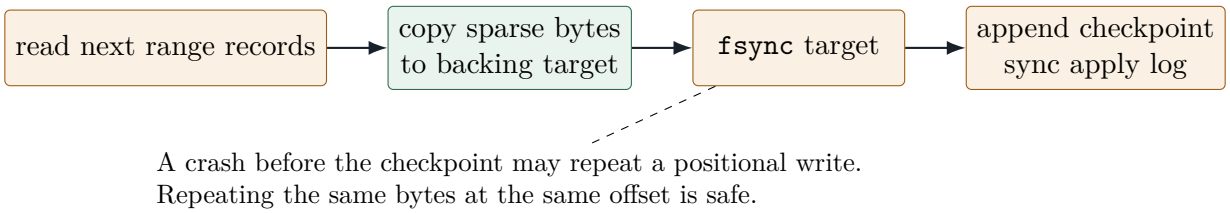

Figure 5: Ordering for one durable apply checkpoint.

`max_files` limits work in one call. The next call scans from the start and uses checkpoints to skip completed data. `remove_applied` first writes a zero checkpoint, then removes session files. This allows a later mount to create a new journal with the same path identifier.

13 Locking and critical sections

Lock	Protects	Not held during
Trie read-write lock	binary trie and policy transactions	ordinary I/O using a handle snapshot
Filesystem inode-map mutex	path map publication and reference detach	backing and overlay data I/O
Inode mutex	passthrough backing identifier and last-open transition	file reads and writes
Inode overlay mutex	range append order across opens	raw and passthrough I/O
Handle mutex	ask resolution and handle-local range refresh	operations on other handles
Session mutex	path registration and apply session consistency	normal raw filesystem I/O
Mount mutex	active session pointer and apply versus mount exclusion	the running FUSE loop

Table 3: Synchronization domains.

The inode-map critical section contains no disk I/O. Policy lookup locks are not used on the ordinary handle fast path. Overlay serialization is limited to writes for the same inode. Apply holds the session mutex because it requires a stable view of path and range metadata.

14 Driver lifecycle

A typical embedding sequence is:

```
var driver: permfuse.Driver = undefined;
try driver.init(io, .{
    .backing_path = "/srv/root",
    .policy_path = "/run/app/policy.trie",
    .state_path = "/run/app/session",
    .passthrough = true,
    .ask_fn = ask,
});
defer driver.deinit();
```

```
try driver.replacePolicyFromText(allocator, fs_block);
try driver.mount(allocator, "/run/app/mount", .{});
```

`mount` is blocking. Another task or thread can call `requestUnmount`. The driver clears the active session pointer only after the loop leaves. Overlay application takes the mount mutex and rejects an active mount, so sparse files cannot be applied while handles may still write them.

15 Error handling and logging

Raw Linux syscall wrappers return encoded syscall results. Those values are decoded with `linux.errno(rc)`. libc and `std.posix` calls use `std.posix.errno(rc)`. The return value is captured once before cleanup can replace thread-local errno.

Errors that prevent a correct reply are returned to FUSE or to the public API. Cleanup and persistence errors that occur after a structure is published are logged. Published trie nodes are not freed in an error cleanup path.

Logging is compiled through a small module adapter. Test and mount roots supply the logger. Messages include source file, line, column, and function.

16 ABI boundary

Zig's C translation treats some libfuse bitfield structures as opaque. The project mirrors the required `fuse_file_info` layout in a packed Zig structure and checks its 64-byte size at compile time. Accessor functions read and write the file handle, cache flags, and passthrough backing identifier.

The code also mirrors the initial fields of `fuse_conn_info` to set the maximum backing stack depth required by passthrough. These definitions are tied to the selected libfuse API version, 318.

17 Testing

The build uses a custom test runner so trie and FUSE logging is available under test. The normal validation commands are:

```
zig build test
zig build test -Doptimize=ReleaseSafe
zig build mount-test -Doptimize=ReleaseSafe
```

Tests cover radix insertion and deletion shapes, free-list reuse, mapping growth, endian conversion, concurrent readers and writers, exact and inherited policy lookup, text round trips, malformed text, path joining, ask approval, inode lifetime, overlay session resume, bounded apply, and torn apply-log recovery.

The interactive `permfuse` command is installed and compiled with the LLVM backend. A separate manual mount target remains as a non-installed integration harness.

18 Limits and operational requirements

- The implementation is Linux-specific and links libc because libfuse requires it.

- Passthrough needs kernel support, `CONFIG_FUSE_PASSTHROUGH`, and suitable privilege such as `CAP_SYS_ADMIN`.
- A policy change does not alter permissions already stored in a non-ask open handle.
- A valid text import can commit some rules if a later allocation or mapping growth fails. Syntax errors are detected before replacement.
- The binary pool has a maximum size below 16 GiB because node indices are 24 bits and each node is 1 KiB.
- Overlay range records use native-endian 64-bit fields. The session format marker must change if this representation changes.

19 Public API summary

19.1 `permtrie`

Function	Purpose
`init`, `deinit`	own the binary file descriptor and mapping
`get`, `getExact`	inherited and exact lookup
`add`, `del`, `reset`	exclusive mutation
`lockWrite`, `unlockWrite`	grouped policy transaction
`getLocked`, `getExactLocked`	lookup inside a writer group
`addLocked`, `delLocked`	mutation inside a writer group
`toText`, `replaceFromText`	`fs` block conversion

19.2 `permfuse.Driver`

Function	Purpose
`init`, `deinit`	own policy, backing descriptor, and session
`getRule`, `evaluate`	exact and inherited policy lookup
`setRule`, `removeRule`	one-rule mutation
`updateRules`	grouped mutation under one writer lock
`policyToText`, `replacePolicyFromText`	policy text conversion
`mount`, `requestUnmount`	run and stop the FUSE loop
`applyOverlay`	resume and apply durable sparse writes

20 Conclusion

permbox-fuse3 separates persistent policy, open-handle decisions, backing I/O, and recoverable overlay data. The trie makes policy lookup compact and keeps updates under one read-write lock. File handles carry a one-byte snapshot for the common path. Kernel passthrough removes FUSE callbacks for handles that need no policy work. Restricted writes remain in sparse files with exact range records and durable apply checkpoints. These parts let the embedding application change policy, answer permission requests, resume sessions, and apply selected writes without turning the library into a separate daemon.